

OCR for Bodo Script: A Comparative Study of Tesseract and TrOCR

A Project Work Submitted in Partial Fulfillment
of the requirements for the Degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Sourjyadyuti Narzary (Roll No.202202021009)

&

Mwikhwam Basumatary (Roll No.202202021021)

&

Dhrub Kumar Basumatary (Roll No.202202021034)

&

Sunjiw Borgoyary (Roll No.202202021017)

Under the supervision of
Dr. Sanjib Narzary



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

केंद्रीय प्रौद्योगिकी संस्थान कोकराझार

CENTRAL INSTITUTE OF TECHNOLOGY, KOKRAJHAR

(A Centrally Funded Institute under Ministry of HRD, Govt. of India)

BODOLAND TERRITORIAL AREAS DISTRICTS :: KOKRAJHAR ::

ASSAM :: 783370

Website: www.cit.ac.in

MAY 2026



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

केंद्रीय प्रौद्योगिकी संस्थान कोकराझार

CENTRAL INSTITUTE OF TECHNOLOGY, KOKRAJHAR

(An Autonomous Institute under MHRD)

Kokrajhar-783370, BTAD, Assam, India

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this project entitled *OCR for Bodo Script: A Comparative Study of Tesseract and TrOCR*, submitted by Sourjadyuti Narzary, Mwikhwam Basumatary, Dhruv Kumar Basumatary & Sunjiw Borgoyary to the Department of Computer Science & Engineering, has been carried out under our direct supervision and guidance.

The project work has been prepared as per the regulations of Central Institute of Technology, and we strongly recommend that it be accepted in partial fulfilment of the requirement for the degree of B.Tech in Computer Science and Engineering.

(Signature)

Supervisor

Name: Dr. Sanjib Narzary

(Assistant Professor)

Dept. of Computer Science & Engineering

CIT Kokrajhar

(Signature)

Head of the Department

Name: Dr. Apurbalal Senapati

(Professor)

Dept. of Computer Science & Engineering

CIT Kokrajhar



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

केंद्रीय प्रौद्योगिकी संस्थान कोकराझार

CENTRAL INSTITUTE OF TECHNOLOGY, KOKRAJHAR

(An Autonomous Institute under MHRD)

Kokrajhar-783370, BTAD, Assam, India

CERTIFICATE BY THE BOARD OF EXAMINERS

This is to certify that the project work entitled *OCR for Bodo Script: A Comparative Study of Tesseract and TrOCR*, submitted by Sourjyadyuti Narzary, Mwikhwim Basumatary, Dhruv Kumar Basumatary & Sunjiw Borgoyary to the Department of Computer Science and Engineering of Central Institute of Technology Kokrajhar, has been examined and evaluated.

The project work has been prepared as per the regulations of Central Institute of Technology and qualifies to be accepted in partial fulfilment of the requirement for the degree of B.Tech in Computer Science and Engineering.

Project Co-ordinator / Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all the individuals who have contributed to the successful completion of our project work. Their guidance, support, and encouragement have been invaluable throughout this journey.

First and foremost, we would like to thank our supervisor Dr. Sanjib Narzary, Dept. of CSE, Asst. Professor, for his unwavering support and expertise. His guidance, constructive feedback, and mentorship have been instrumental in shaping this project. We are truly grateful for his patience, dedication, and valuable insights that have significantly enhanced the quality of our work.

Furthermore, we extend our gratitude to our fellow students who have provided valuable input, suggestions, and motivation during the course of this project. Their collaborative spirit and willingness to share their ideas have significantly enriched our work.

Lastly, we would like to express our appreciation to anyone else who has contributed to this project in any way, whether directly or indirectly. Their support, feedback, and encouragement have played a significant role in completing this endeavour.

Date:

Sourjyadyuti Narzary (202202021009)

Mwikhwm Basumatary (202202021021)

Dhrub Kr. Basumatary (202202021034)

Sunjiw Borgoyary (202202021017)

ABSTRACT

Bodo is a low-resource scheduled language of India, spoken predominantly by the Bodo community of Assam and other parts of Northeast India. Despite being an Eighth Schedule language of the Indian Constitution, Bodo remains poorly served in the domain of digital language technology, with no dedicated and reliable Optical Character Recognition System available for practical use. The absence of such a system limits the ability to digitise and preserve Bodo language content that currently exists only in physical form.

This project presents a comparative study of two OCR approaches fine-tuned specifically for Bodo script recognition: the classical LSTM-based Tesseract OCR engine and the modern transformer-based TrOCR model developed by Microsoft Research. For the Tesseract pipeline, a synthetic dataset of over 5,000 training images was generated using a custom Python automation pipeline that rendered Bodo text from a corpus collected from AI4Bharat, Bodo news websites, and a digital dictionary. For the TrOCR pipeline, a real Bodo script dataset of approximately 28,000 samples was collected from the Bodosa newspaper, Bodo books, Bodo book covers, and raw handwritten samples. From this pool, a curated subset of 5,000 samples was selected for fine-tuning to keep training tractable within the available compute budget while preserving balanced coverage across the four sources.

Experimental results demonstrate that while Tesseract produced satisfactory results on printed Bodo text, it struggled significantly with handwritten Bodo script. TrOCR, by contrast, demonstrated a superior ability to recognise both printed and handwritten Bodo text, even when trained on only 5,000 samples out of the available 28,000-sample dataset. These findings establish TrOCR as the more capable and generalisable approach for Bodo script OCR, particularly for real-world scenarios involving handwritten content. This work represents one of the first systematic comparative OCR studies for the Bodo language and contributes a curated Bodo script dataset, a fine-tuned TrOCR model, and baseline evaluation metrics to support future research in Bodo language digitisation and natural language processing.

TABLE OF CONTENTS

1	Introduction	1
1.1	Motivation	1
2	Related Work	3
2.1	OCR for Devanagari and Indian Scripts	3
2.2	Tesseract OCR for Low-Resource Languages	3
2.3	Synthetic Dataset Generation for OCR	3
2.4	Post-OCR Error Correction for Bodo and Devanagari Languages	3
3	Proposed Methodology	5
3.1	Overview	5
3.2	Proposed System Design	5
3.3	Overall Workflow Diagram	6
3.4	Dataset Description	6
3.4.1	Overview	6
3.4.2	Dataset Preparation of Tesseract OCR	7
3.4.3	Dataset Preparation of TrOCR	7
3.4.4	Architecture of the Tesseract OCR Engine	8
3.4.5	Tesseract Fine-Tuning for Bodo Script	11
3.4.6	Architecture of the TrOCR Model	16
3.4.7	Proposed Algorithm and Design for Bodo OCR	17
3.4.8	Understanding the TrOCR Architecture: A Stage-by-Stage Explanation	18
3.5	Fine-Tuning Procedure	20
4	Experimental Results	23
4.1	Experimental Results of Tesseract OCR	23
4.1.1	Tesseract Failure on Handwritten Input	25
4.2	Experimental Results for TrOCR	25
4.3	Evaluation Metrics	25
4.4	Training Loss Analysis	26
4.5	Qualitative Results of TrOCR	27
4.6	Discussion	28
4.7	Comparative Analysis: Tesseract vs TrOCR	29
4.7.1	Quantitative Comparison: Tesseract vs TrOCR	29
5	Conclusion	32
5.1	Drawbacks	32
6	Future Work	33
	Bibliography	33

LIST OF FIGURES

3.1	Overall Workflow of the Proposed OCR System	6
3.2	Tesseract System Architecture.	9
3.3	Tesseract LSTM Training Plot for the Bodo (bxx) model — Character Error Rate (CER) over training iterations, showing both training and validation curves converging toward the target error rate.	13
3.4	Input Bodo script image (left) and the corresponding Tesseract OCR output (right) produced by the fine-tuned bxx model.	14
3.5	Architecture of TrOCR Model	15
3.6	End-to-end TrOCR pipeline fine-tuned for Bodo script recognition.	18
4.1	Tesseract OCR Result — Sample 1: Short Bodo sentences (printed script).	23
4.2	Tesseract OCR Result — Sample 2: Multi-line Bodo paragraph (printed script).	23
4.3	Tesseract OCR Result — Sample 3: Long-form Bodo text passage (printed script).	24
4.4	Tesseract failure case: the fine-tuned bxx model produces no meaningful output when given a handwritten Bodo script image, highlighting its limitation to printed synthetic text only.	25
4.5	TrOCR fine-tuning loss curve for the Bodo OCR model across all three training stages (25 total epochs).	26
4.6	TrOCR result on a Bodo book cover. The Kaggle notebook on the left shows the predicted Bodo text; the image on the right is the original input.	27
4.7	TrOCR result on a raw handwritten Bodo sample. The notebook on the left shows the predicted Bodo text; the scan on the right is the original handwritten input.	27
4.8	TrOCR result on a multi-line Bodo book cover. The notebook on the left shows the predicted Bodo text; the cover on the right is the original input.	28
4.9	TrOCR result on a stylised Bodo book cover. The notebook on the left shows the predicted Bodo text; the cover on the right is the original input.	28
4.10	Comparative performance of Tesseract and TrOCR for Bodo OCR. Left: CER comparison — Tesseract achieved 4.73% CER on single-line printed text while TrOCR’s CER is not directly comparable due to multi-line annotation mismatch. Right: qualitative capability comparison across five evaluation dimensions based on experimental observations.	30
4.11	Bodo OCR dataset composition. Left: distribution of the four source types in the TrOCR real-world dataset (28,000 images). Right: training, validation, and test split comparison between the Tesseract synthetic pipeline and the TrOCR real-world pipeline.	30

LIST OF TABLES

3.1	Fine-Tuning Hyperparameters	20
3.2	Stage 1 (5 Warm-up Epochs)	21
3.3	Stage 2 Loss Training History	21
3.4	Stage 3 Loss History (10 Fine Refinement Epochs)	22
4.1	Summary of Training Loss Across All Three Stages	27
4.2	Overall Comparison: Tesseract vs TrOCR for Bodo OCR	29
4.3	CER and WER on Bodo Test Set (100 samples)	29

1. INTRODUCTION

The rapid growth of digital technologies has made the digitisation of printed and handwritten documents essential for education, research, and the preservation of linguistic heritage. Optical Character Recognition (OCR) plays a crucial role in converting scanned text images into editable and searchable digital text. While several robust OCR systems exist for widely used languages such as English, Hindi, and Assamese, the Bodo language continues to be underrepresented in digital tools and technologies [7].

Bodo (Boro) is a Tibeto-Burman language with over 1.5 million speakers according to the 2011 Census of India [17], spoken primarily by the Boro people of Assam’s Bodoland Territorial Region in Northeast India. It is recognised as an associate official language of Assam and as one of the 22 Eighth Schedule languages of the Indian Constitution, having been added to the Eighth Schedule in 2003 [18]. The language has been written in the Devanagari script since 1975 [15]. However, despite its cultural and linguistic importance, Bodo still lacks a reliable OCR system capable of accurately recognising its script. Existing OCR engines — including popular models such as Tesseract and recent AI-based OCR tools — perform poorly when tested on Bodo text. Errors in character recognition and inaccurate segmentation make these tools unsuitable for practical use [7].

This absence of OCR support presents a significant challenge for digitising Bodo books, newspapers, educational materials, and official documents. Without effective OCR support, Bodo-speaking communities face difficulties in creating digital archives, developing language technologies, and promoting the use of their language in digital environments [16].

To address this issue, our project focuses on developing a specialised OCR system for the Bodo language, supported by a custom dataset collected from books, press materials, handwritten samples, and other real-world sources [15]. The aim is to improve text recognition accuracy and enable reliable digitisation of Bodo content, thereby contributing to linguistic preservation and technological inclusion.

1.1 MOTIVATION

The Bodo language, a low-resource language spoken predominantly in the Bodoland Territorial Region of Assam, faces significant challenges in the digital domain. Despite having a rich literary and cultural heritage, Bodo text (written in the Devanagari script) remains largely inaccessible in digital form. A vast amount of valuable material lies in old books, newspapers, and printed documents that have not yet been digitised.

Optical Character Recognition (OCR) technology has proven to be a transformative tool for document digitisation, widely adopted by organisations such as Google, Microsoft, ABBYY, and Adobe. However, existing OCR systems predominantly support well-resourced languages and fail to adequately serve minority languages like Bodo. The scarcity of annotated datasets and the absence

of dedicated OCR tools further widen this digital gap.

This project is motivated by the need to bridge that gap by developing a functional OCR model capable of recognising Bodo Devanagari text from images and converting it into editable digital form. This work aims to lay a foundation for a broader, community-driven effort to preserve, document, and make accessible the Bodo language for future generations.

OBJECTIVES

1. Develop a Bodo OCR model by fine-tuning the Tesseract OCR engine to recognise Bodo Devanagari text from images.
2. Create a synthetic dataset pipeline to generate training data using multiple fonts and noise variations, overcoming the lack of real-world Bodo text datasets.
3. Evaluate model performance by measuring Character Error Rate before and after fine-tuning.
4. Enable digitisation of old Bodo books and printed documents to preserve the language's literary heritage.
5. Make the tool accessible for practical use by Bodo language speakers and the general public.
6. Release the dataset publicly to support future research and community-driven development of Bodo language tools.

2. RELATED WORK

2.1 OCR FOR DEVANAGARI AND INDIAN SCRIPTS

OCR research on Indian languages has concentrated mostly on widely spoken languages such as Hindi, Bengali, and Marathi. A detailed survey covering OCR in Bangla and Devanagari scripts reviewed the major techniques and their results, and confirmed that Devanagari is one of the two most studied Indian scripts [6]. For Devanagari-based languages such as Bodo, Maithili, Konkani, Dogri, and Kashmiri, however, OCR research remains limited because of the distinct challenges these languages pose [7]. This is the gap that motivates the present work on Bodo OCR.

2.2 TESSERACT OCR FOR LOW-RESOURCE LANGUAGES

Tesseract has been applied successfully to many under-resourced languages thanks to its open-source LSTM model. A study on Kurdish OCR showed that using a training dataset with scripts similar to the target low-resource language can substantially boost the performance of the Tesseract LSTM engine: the existing model is reused and then tuned on a small amount of data from the target language [9]. In another study, the Tesseract model was improved for Tamil and Sinhala by tuning the LSTM model on old fonts, lowering the character error rate considerably [11]. For Arabic, fine-tuning the LSTM model on a wide range of fonts reduced the Character Error Rate (CER) by up to 61% and the Word Error Rate (WER) by up to 70% [10].

2.3 SYNTHETIC DATASET GENERATION FOR OCR

The scarcity of annotated real-world data for low-resource languages has driven researchers toward synthetic data generation. SynthOCR-Gen, an open-source synthetic OCR dataset generator, transforms digital Unicode text corpora into ready-to-use training datasets, addressing the high cost and effort of manual dataset creation [14]. A similar approach was applied in Indic language OCR, where models trained on synthetic data and fine-tuned on real data achieved high accuracy [13]. In this project, a custom Python pipeline was used to generate over 5,000 synthetic Bodo training images with noise augmentation to simulate real-world conditions.

2.4 POST-OCR ERROR CORRECTION FOR BODO AND DEVANAGARI LANGUAGES

RoundTripOCR, proposed by researchers at IIT Bombay, introduced a synthetic data generation approach to address the scarcity of post-OCR correction datasets for low-resource languages. They released datasets for Hindi, Marathi, Bodo, Nepali, Konkani, and Sanskrit [8]. The inclusion of Bodo

confirms its recognition as an underserved language in OCR research. This work also highlights that post-OCR correction remains necessary, emphasising the need for a high-accuracy base OCR model.

3. PROPOSED METHODOLOGY

3.1 OVERVIEW

This project presents a comparative study of Tesseract OCR and TrOCR for recognising Bodo script written in Devanagari. Both systems were implemented and evaluated separately. The main objective of the proposed methodology is to analyse the effectiveness of traditional OCR techniques and transformer-based OCR models for low-resource language recognition.

3.2 PROPOSED SYSTEM DESIGN

The proposed system consists of dataset preparation, OCR implementation, prediction generation, and comparative performance evaluation. Separate workflows were designed for Tesseract OCR and TrOCR so that their performance could be analysed individually.

3.3 OVERALL WORKFLOW DIAGRAM

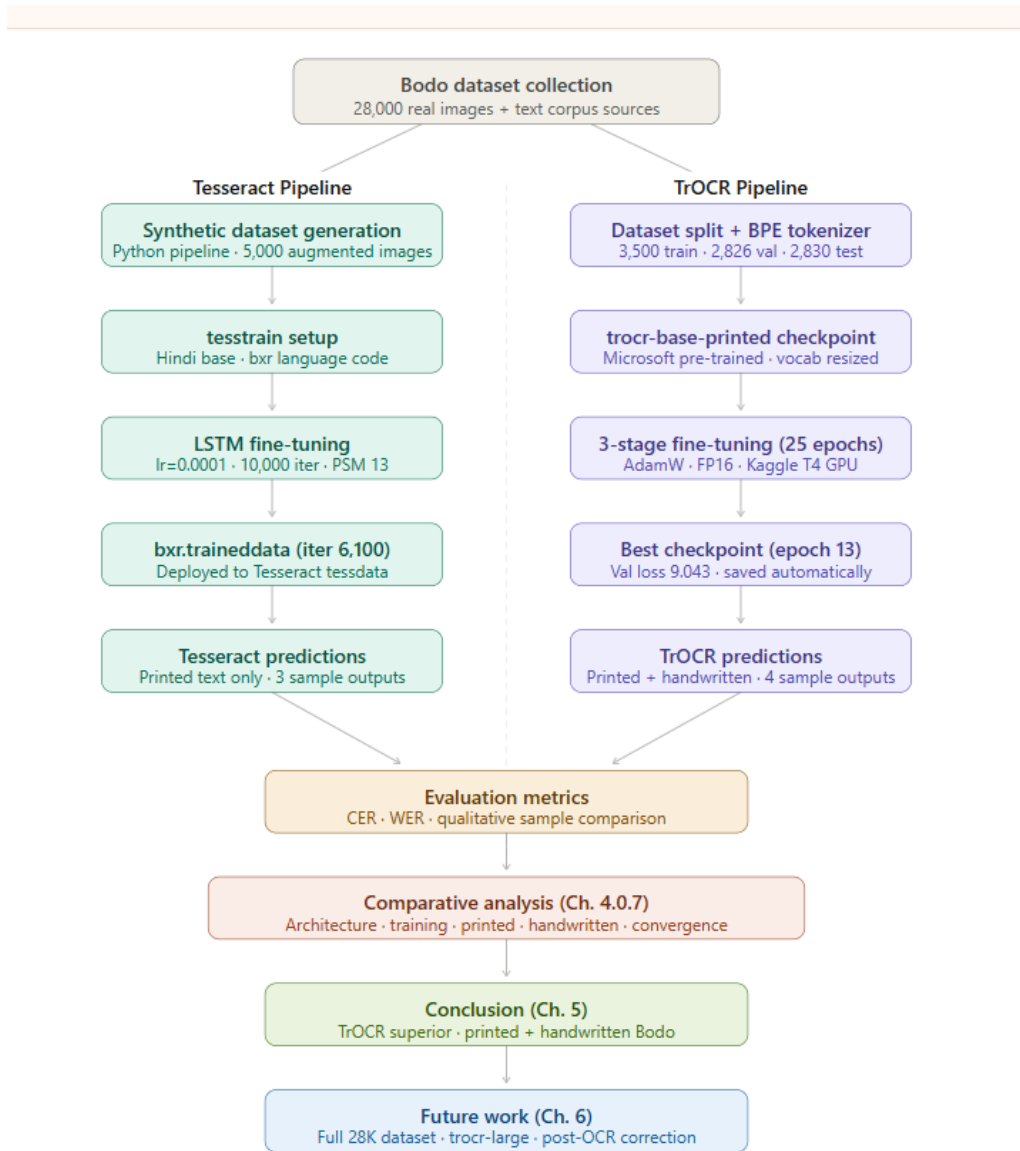


Figure 3.1: Overall Workflow of the Proposed OCR System

3.4 DATASET DESCRIPTION

3.4.1 Overview

Bodo is a low-resource language, and sufficient quality text data is difficult to obtain in physical form. A synthetic dataset approach was therefore the more feasible option for our work.

For the TrOCR model, a custom Bodo script dataset was assembled. The full dataset comprises approximately 28,000 annotated image–text pairs, making it one of the more structured datasets created exclusively for Bodo script OCR research. From this pool, 5,000 samples were used for the training phase, with the remainder reserved for future work in order to keep training tractable on the available hardware.

For Tesseract OCR, a text corpus was collected from several sources, including the *Abithai* app-source dataset, Bodo news websites, and a digital dictionary. After the raw text was collected and consolidated into a corpus, synthetic augmented images were generated for training using a Python augmentation pipeline. The generated images simulated real-world degradation such as old-newspaper artefacts, ink-style imperfections, and random distortions, exposing the model to a wide range of input conditions.

3.4.2 Dataset Preparation of Tesseract OCR

Source: *Abithai* app-source dataset, Bodo news websites, and a digital dictionary.

After the raw text was collected and consolidated into a corpus, synthetic augmented images were generated for training. A Python augmentation pipeline was developed to produce these training images with noise augmentations; the noise augmentations improve robustness by simulating real-world image degradation.

Pipeline:

- **Raw text file:** consolidates all collected text into a single source for image rendering.
- **Clean image generator:** renders each text line as an image containing between 2 and 5 words, varying the font, colour style, and font size (2–4 pt). Corresponding word- and line-level annotations are written out to a TSV file.
- **Noise adder:** a separate script that applies random augmentations to each generated image, including Gaussian noise, blur, and brightness variation.

This pipeline produced over 5,000 synthetic training images.

3.4.3 Dataset Preparation of TrOCR

The TrOCR dataset is organised into two main folders: **Photos** and **Annotations**.

Photos folder

The Photos folder contains all raw image data collected from four distinct sources:

Source	Type	Description
Bodosa Newspaper	Printed text	Bodosa newspaper pages downloaded from three websites and cropped into small line-level portions.
Raw handwritten data	Handwritten	Handwritten Bodo script collected from students, covering natural writing styles and variations.
Bodo book covers	Printed text	Text captured from the cover pages of various Bodo books.
Bodo books	Printed text	Body text, paragraphs, and headings scanned from Bodo language books.

Annotations folder

The Annotations folder contains three JSON files:

train.json: annotations for the training data, containing about 3,500 samples.

val.json: annotations for the validation data, containing about 2,500 samples.

test.json: annotations for the test data, containing about 2,500 samples.

3.4.4 Architecture of the Tesseract OCR Engine

Tesseract follows a traditional multi-stage pipeline architecture, though several of its stages incorporate design choices that were notably unconventional at the time of its development and, in some respects, remain so today.

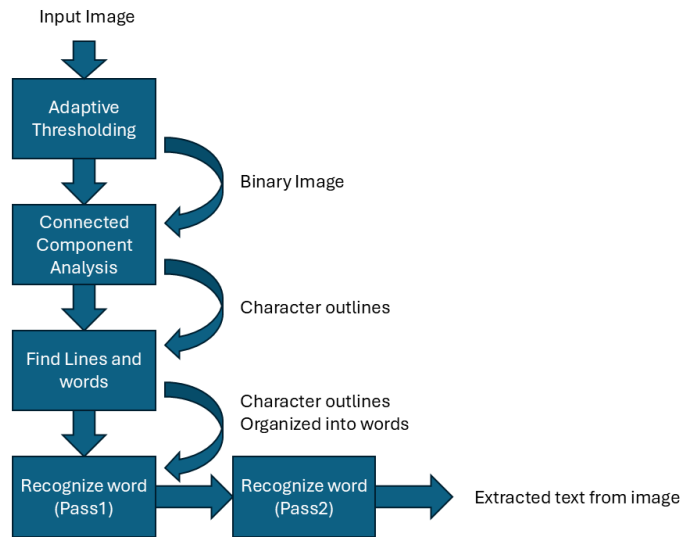


Figure 3.2: Tesseract System Architecture.

Input and Preprocessing

Tesseract assumes its input to be a binary image, optionally accompanied by polygonal text region definitions. Unlike many OCR systems of its era, Tesseract was developed independently of page layout analysis, relying instead on external tools for that function. The first structural step involves a connected component analysis, in which the outlines of components are stored rather than the components themselves. Though computationally expensive, this design decision enabled a significant advantage: by examining the nesting of outlines and the number of child and grandchild outlines, the engine can trivially detect and process inverse (white-on-black) text — a capability that was pioneering at the time.

Blob and Line Organisation

Following connected component analysis, outlines are gathered into structures called *Blobs* based purely on nesting relationships. These Blobs are subsequently organised into text lines, and both the lines and surrounding regions are analysed to determine whether the text follows fixed-pitch or proportional spacing.

- **Fixed-pitch text** is immediately segmented into characters using the detected pitch.
- **Proportional text** is broken into words using a combination of definite and fuzzy spaces, with ambiguous cases deferred for resolution after word recognition.

Line Finding and Baseline Fitting

The line-finding algorithm is designed to recognise skewed pages without requiring deskewing, thereby preserving image quality. It operates through blob filtering — using a percentile height filter to remove noise, punctuation, and drop-caps — followed by x -coordinate-sorted blob assignment to unique text lines while tracking slope across the page. A least median of squares fit is then applied to estimate baselines, after which filtered-out blobs are reintegrated into the appropriate lines.

A particularly notable contribution is Tesseract's use of **quadratic spline fitting** for baseline estimation. This enabled the engine to handle curved baselines — a frequent artefact of document scanning — and represented a first for OCR systems at the time.

Word Recognition: A Two-Pass Approach

Recognition in Tesseract proceeds through a two-pass process:

1. **First Pass:** Each word is recognised in sequence. Words recognised with sufficient confidence are passed to an adaptive classifier as training data.
2. **Second Pass:** Words that were not adequately recognised in the first pass are reprocessed, benefiting from patterns learned by the adaptive classifier during the first pass.

A final phase resolves fuzzy spaces and evaluates alternative hypotheses regarding x -height, enabling the detection of small-cap text.

Character Segmentation

For non-fixed-pitch text, word recognition involves iterative character segmentation through two mechanisms:

- **Chopping:** Blobs with the lowest classifier confidence are chopped at candidate points derived from concave vertices of a polygonal approximation. Up to three pairs of chop points may be used to separate joined characters.
- **Association:** If chopping is insufficient, an A* (best-first) search is performed over the segmentation graph of maximally chopped blobs to identify the optimal combination of fragments into candidate characters.

Static and Adaptive Classifiers

Tesseract employs a dual-classifier strategy.

Static Classifier. The static classifier uses small, fixed-length features extracted from character outlines, matched many-to-one against clustered prototype features derived from training data. This asymmetric matching — small features against larger prototypes — confers robustness to damaged or broken characters. Classification itself is a two-step process: a class pruner first generates a shortlist of candidate character classes, followed by detailed similarity computation against prototype configurations.

Adaptive Classifier. The adaptive classifier shares the same feature set and classifier architecture as the static classifier but differs in its normalisation strategy. While the static classifier uses centroid and moment-based normalisation, the adaptive classifier employs isotropic baseline/ x -height normalisation, which better distinguishes uppercase from lowercase characters and improves robustness to noise. Trained incrementally during the first recognition pass, the adaptive classifier refines discrimination within a document by exploiting the limited font variability typical of individual documents.

Linguistic Analysis

Tesseract’s linguistic processing is intentionally minimal. A module called the *permuter* selects the best candidate word string from several categories — including frequent words, dictionary words, numeric strings, and classifier-preferred outputs — by assigning each a weighted distance score. Word comparisons across segmentations of differing character counts are normalised using two metrics:

- **Confidence score:** the negative normalised prototype distance.
- **Rating:** the normalised distance scaled by total outline length, which is summable across characters within a word.

3.4.5 Tesseract Fine-Tuning for Bodo Script

Setup for Training

To begin, the `tesstrain` repository was cloned from <https://github.com/tesseract-ocr/tesstrain> into the project directory. The following dependencies were then installed:

1. Latest Tesseract (e.g. from <https://digi.bib.uni-mannheim.de/tesseract/>)
2. Python 3
3. Git SCM for Windows — this package supplies many Linux-style utilities for Windows environments (e.g. `find`, `unzip`, `rm`)
4. Windows Package Manager
5. Required Python libraries: `Pillow`, `python-bidi`, `matplotlib`, `pandas`

Tesseract v5 leverages GNU make to expose a comprehensive set of targets and configurable variables for the training workflow.

Ground truth data — comprising line images paired with their transcriptions — was organised into the `data/MODEL_NAME-ground-truth` directory within `tesstrain`. These files are automatically partitioned into training and evaluation subsets according to the `RATIO_TRAIN` parameter. The model was assigned the identifier `bxr`, corresponding to the ISO 639-3 language code for Bodo: <https://iso639-3.sil.org/code/bxr>.

A separate `tessdata` directory was created at the same level as `tesstrain`, into which the pre-trained Hindi model file (`hin.traineddata`) was placed as the fine-tuning base.

Training Process

The following `make` command was used to initiate training, with the parameters configured as shown:

```
make training \
    MODEL_NAME=bxr \
    START_MODEL=hin \
    TESSDATA=../tessdata/ \
```

```
MAX_ITERATIONS=10000 \  
LEARNING_RATE=0.0001 \  
LANG_TYPE=Indic \  
PSM=13 \  
RATIO_TRAIN=0.90 \  
TARGET_ERROR_RATE=0.01
```

Each parameter is described below:

- **MODEL_NAME** — specifies the name of the output model.
- **START_MODEL** — the base model from which training continues (fine-tuning source).
- **TESSDATA** — path to the directory holding the **START_MODEL** file.
- **MAX_ITERATIONS** — upper limit on the number of training iterations.
- **LEARNING_RATE** — controls the step size during gradient descent.
- **LANG_TYPE** — specifies the script family (e.g. `Indic`, `RTL`, or blank for `LTR`).
- **PSM** — sets the page segmentation mode for Tesseract.
- **RATIO_TRAIN** — defines the proportion of data allocated to training versus evaluation.
- **TARGET_ERROR_RATE** — training halts automatically once the character error rate (CER, in percent) drops below this threshold.

Training Plot

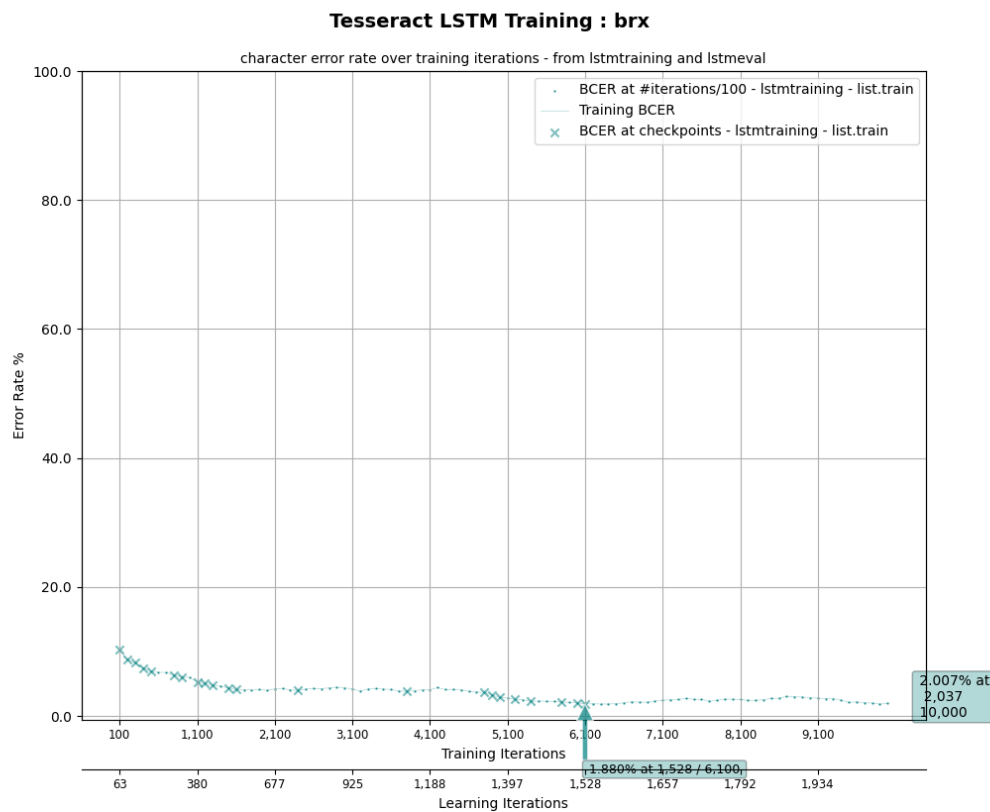


Figure 3.3: Tesseract LSTM Training Plot for the Bodo (bxx) model — Character Error Rate (CER) over training iterations, showing both training and validation curves converging toward the target error rate.

Throughout the training process, intermediate LSTM checkpoint files were generated at regular intervals. Upon completion, the following output files were produced:

- **bxx.unicharset** — the full character inventory from the dataset.
- **bxx.puncs** — a punctuation reference file.
- **bxx.wordlist** — a deduplicated vocabulary list.
- **bxx.traineddata** — the final trained model.

The final model was extracted from the checkpoint at iteration 6100 using the `lstmtraining` utility with the following command:

```
lstmtraining \
  --stop_training \
  --continue_from 6100 \
  --traineddata data/bxx/bxx.traineddata \
  --model_output bxx.traineddata
```

To deploy the model, the generated `bxr.traineddata` file was copied into the Tesseract installation's `tessdata` directory at `C:\Program Files\Tesseract-OCR\tessdata`. The model was then evaluated on a sample image via Git Bash using the following command: `tesseract output.jpeg output -l bxr`

Evaluation on Sample Bodo Image

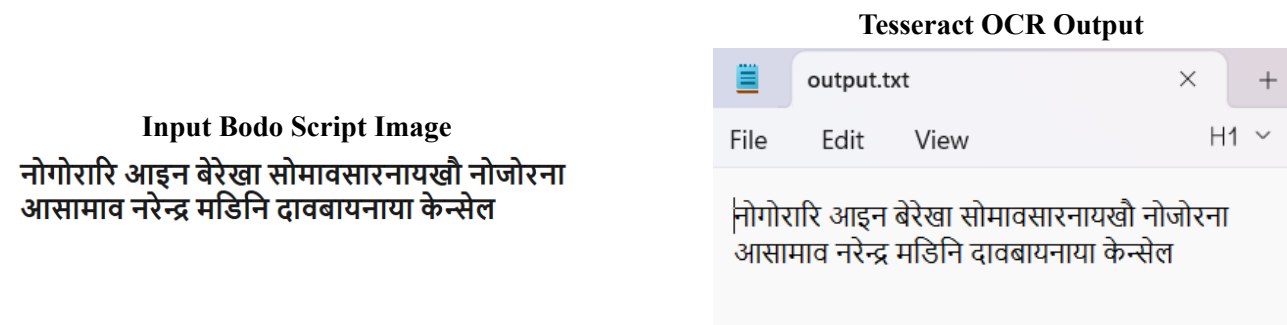


Figure 3.4: Input Bodo script image (left) and the corresponding Tesseract OCR output (right) produced by the fine-tuned `bxr` model.

Architecture of TrOCR Model

The second approach adopted in this study involves fine-tuning the TrOCR (Transformer-based Optical Character Recognition) model developed by Microsoft Research for the recognition of Bodo script. TrOCR is a state-of-the-art end-to-end Transformer-based OCR model for text recognition with pre-trained computer vision and NLP models [1]. It combines a Vision Transformer (ViT) encoder with a RoBERTa-style autoregressive decoder [3]. Unlike classical OCR engines such as Tesseract, TrOCR does not require explicit character segmentation, hand-crafted feature extraction, or rule-based layout analysis. Instead, it learns to directly map raw image pixel information to Unicode text sequences through an attention-based encoder-decoder mechanism [5, 1].

TrOCR represents a significant advancement in OCR technology and can accurately predict texts against various kinds of image effects [2]. The model is publicly documented and accessible through the Hugging Face Transformers library [4].

This TrOCR is built on a standard encoder-decoder Transformer architecture which consists of two pre-trained components that are fine-tuned for the OCR task for the Bodo language [1, 3].

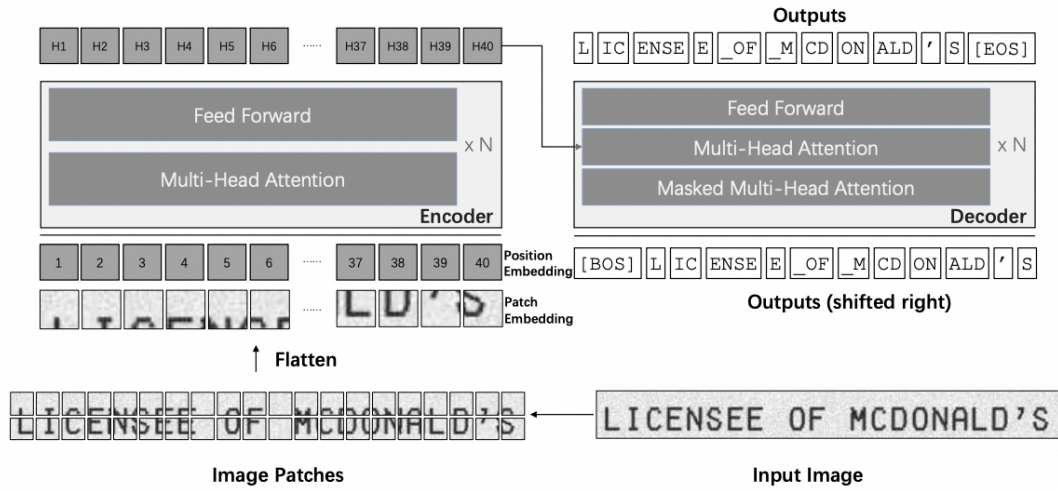


Figure 1: Model Architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

Figure 3.5: Architecture of TrOCR Model

3.4.6 Architecture of the TrOCR Model

The model has two principal components: a Vision Transformer (ViT) that converts a raw image into a sequence of contextual patch embeddings, and a RoBERTa-style decoder that converts those embeddings into a Unicode character sequence one token at a time. We describe each component in detail below.

Encoder: Vision Transformer (ViT)

The ViT encoder [3] treats an image the same way a standard Transformer treats a sentence: it divides the input into pieces, embeds each piece, and then lets attention layers discover the relationships among them. The key difference from text is simply *what* gets embedded — image patches rather than word tokens.

Preprocessing and patch extraction. Every input image — whether a scanned newspaper page, a printed document, or a photograph of a handwritten manuscript — is first resized to 384×384 pixels. Each colour channel is then normalised using the ImageNet population statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), producing a floating-point tensor $\tilde{\mathbf{X}} \in R^{3 \times 384 \times 384}$. That tensor is partitioned into non-overlapping square patches of side length $P = 16$ pixels, yielding a total of

$$N = \frac{H \times W}{P^2} = \frac{384 \times 384}{16 \times 16} = 576 \text{ patches.} \quad (3.1)$$

Patch embedding. Each patch $\mathbf{p}_i \in R^{768}$ (obtained by flattening the $16 \times 16 \times 3$ pixel block) is multiplied by a learnable projection matrix $\mathbf{E} \in R^{768 \times d}$ and augmented with a learnable positional embedding $\mathbf{e}_i^{\text{pos}}$:

$$\mathbf{z}_i^{(0)} = \mathbf{p}_i \mathbf{E} + \mathbf{e}_i^{\text{pos}}, \quad i = 1, \dots, 576. \quad (3.2)$$

For the trocr-base model used here, $d = 768$.

Self-attention layers. The 576 embeddings are then processed by $L_E = 12$ identical Transformer encoder blocks. Each block applies multi-head self-attention (MHSA) and a two-layer feed-forward network (FFN), both wrapped with layer normalisation (LN) and residual connections: $\mathbf{Z}'^{(\ell)} = \text{MHSA}(\text{LN}(\mathbf{Z}^{(\ell-1)})) + \mathbf{Z}^{(\ell-1)}$,

$\mathbf{Z}^{(\ell)} = \text{FFN}(\text{LN}(\mathbf{Z}'^{(\ell)})) + \mathbf{Z}'^{(\ell)}$. After all 12 blocks, the output $\mathbf{M} = \mathbf{Z}^{(12)} \in R^{576 \times 768}$ is a matrix of *contextual visual features*. Every row encodes not just its own 16×16 pixel region but also information about every other region, because attention is computed globally across all 576 positions. This global receptive field is especially valuable for Bodo text: matras, half-consonants, and conjunct forms all interact spatially, and only a mechanism that can look at the full image at once can reliably resolve such interactions.

Decoder: Autoregressive RoBERTa Language Model

While the encoder is purely visual, the decoder is purely linguistic. Initialised from a RoBERTa checkpoint, it generates the output text one token at a time, conditioned at every step on what the

encoder found in the image.

Cross-attention bridge. The connection between encoder and decoder is a cross-attention sublayer present inside every decoder block [5]. At generation step t , query vectors $\mathbf{Q}$ are derived from the decoder’s current hidden state, while keys $\mathbf{K}$ and values $\mathbf{V}$ come from the encoder output $\mathbf{M}$:

$$CrossAttn(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = softmax\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (3.3)$$

Because $\mathbf{K}$ and $\mathbf{V}$ are always derived from the full set of 576 patch embeddings, the decoder can attend to any region of the image at any point during generation, giving it character-level visual grounding [1].

Masked self-attention. Inside the decoder, a causal (lower-triangular) attention mask prevents each token position from attending to future tokens, preserving the autoregressive property. With $L_D = 12$ decoder blocks, the hidden state at step t is

$$\mathbf{h}_t = DecoderBlocks_{1:12}(y_{t-1}, \mathbf{M}). \quad (3.4)$$

Token prediction and vocabulary. The hidden state $\mathbf{h}_t$ is projected through an output matrix $\mathbf{W}_o \in R^{d \times |\mathcal{V}|}$, where $|\mathcal{V}| = 5,000$ is our Bodo-specific vocabulary size:

$$P(y_t | y_{<t}, \mathbf{M}) = softmax(\mathbf{W}_o \mathbf{h}_t). \quad (3.5)$$

The predicted token is appended to the sequence and fed back as input at the next step, continuing until an `<EOS>` token is produced or the 64-token limit is reached. At inference time we use beam search with $k = 4$ beams, `no_repeat_ngram_size = 3`, and a repetition penalty of 1.5 to prevent the decoder from cycling through the same syllable repeatedly — a failure mode observed in early training predictions.

Vocabulary adaptation. A critical modification specific to our Bodo work was resizing the decoder’s embedding table from the original English sub-word vocabulary to our 5,000-token Bodo BPE vocabulary. This was done by calling `resize_token_embeddings(5000)` on the decoder after loading the base checkpoint. Without this step the output projection layer has the wrong shape and the model either fails or generates entirely meaningless output.

3.4.7 Proposed Algorithm and Design for Bodo OCR

Figure 3.6 captures the complete seven-stage flow from raw image to predicted Bodo string.

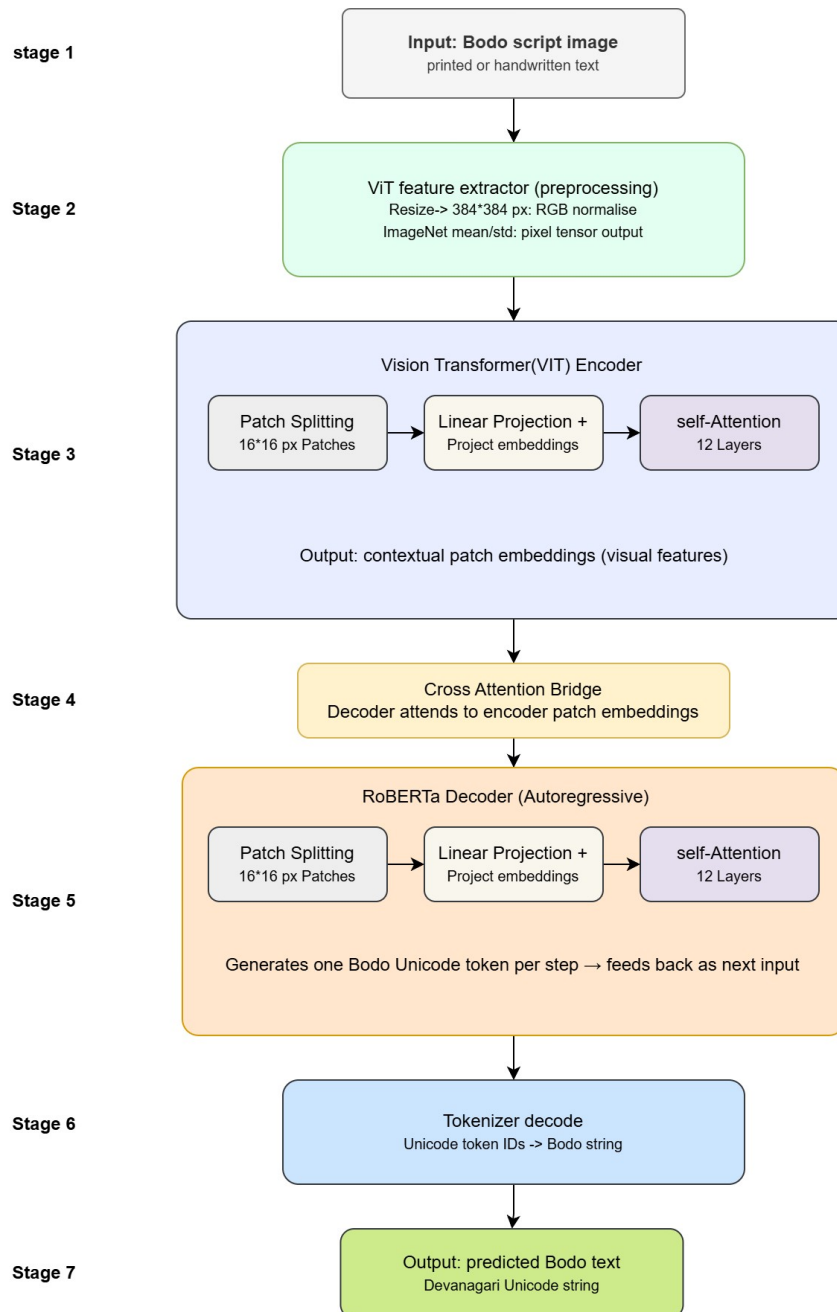


Figure 3.6: End-to-end TrOCR pipeline fine-tuned for Bodo script recognition.

3.4.8 Understanding the TrOCR Architecture: A Stage-by-Stage Explanation

Figure 3.6 shows the complete pipeline of our Bodo OCR system. This subsection walks through each of the seven stages in plain terms, explaining not just *what* happens at every step but *why* it is needed for recognising Bodo script.

Stage 1 — Input: Bodo Script Image

The process begins with a raw photograph or scan that contains Bodo text. The image may come from a printed newspaper, a typeset book page, or a handwritten manuscript. No restrictions are placed on font, resolution, or background colour at this stage — the system is designed to handle both printed and handwritten Bodo script without any manual pre-selection.

Stage 2 — ViT Feature Extractor (Preprocessing)

Before the neural network sees the image, the image needs to be cleaned and standardised. Think of this step as ironing a crumpled sheet of paper so that every word lies flat and is equally readable. Three operations are carried out:

1. **Resize to 384×384 pixels.** Every input image — regardless of its original size — is rescaled to the same fixed dimensions. This ensures that the model always receives input of a known shape and that no patch in the image is accidentally given more weight simply because the original photograph was larger.
2. **Channel-wise colour normalisation.** The brightness values of each colour channel (red, green, blue) are adjusted to a standard range using the statistical parameters learned from the ImageNet dataset. This matters because the Vision Transformer encoder was pre-trained on ImageNet, so its weights expect pixel values in that specific distribution. Without normalisation the model would interpret a well-lit scan and a dim photograph of the same text very differently.
3. **Conversion to a pixel tensor.** The normalised image is stored as a grid of floating-point numbers (a tensor of shape $3 \times 384 \times 384$) that the GPU can process. In our implementation this tensor is kept in half-precision (16-bit) format to halve memory usage on the Kaggle T4 GPU without any meaningful loss in accuracy.

Stage 3 — Vision Transformer (ViT) Encoder

This is the "eyes" of the system. It reads the image and understands what it sees. It works in three steps: Patch Splitting: The 384×384 image is cut into 576 small tiles, each 16×16 pixels — like cutting a photograph into a grid of tiny squares. Linear Projection: Each small tile is converted into a list of numbers (an embedding) that captures what that tile looks like. Position information is also added so the model knows where each tile sits in the original image. Self-Attention (12 Layers): This is the clever part. All 576 tiles look at each other and share information. A tile containing a vowel matra can influence how a nearby consonant tile is understood. After 12 rounds of this, each tile now carries rich information about not just itself but the whole image. The output of Stage 3 is called contextual patch embeddings — basically a deep understanding of every region of the image.

Stage 4 — Cross-Attention Bridge

This is the handover point between the "eyes" (encoder) and the "pen" (decoder). The decoder does not just receive a single summary of the image — instead, at every moment while writing, it can look back at any of the 576 image tiles it needs. It is like a typist who keeps glancing back at the original document while typing, rather than trying to memorise it all at once.

Stage 5 — RoBERTa Decoder (Autoregressive)

This is the "pen" of the system — it writes out the Bodo text one character at a time. It works similarly to the encoder: It takes the previously written character as input. Through its own attention layers it keeps track of everything already written so far. It looks at the image (via Stage 4) to decide what

comes next. It outputs one Bodo Unicode token per step, then feeds that token back as input for the next step. This continues character by character until the full word or line is written out.

Stage 6 — Tokenizer Decode

The decoder produces a sequence of numbers (token IDs), not actual letters. This stage converts those numbers back into real Bodo characters in Devanagari Unicode — the same characters you would see in a Bodo text document.

Stage 7 — Output: Predicted Bodo Text

The final output is a proper Devanagari Unicode string — the recognised Bodo text. This can now be saved, searched, translated, or used in any NLP application.

3.5 FINE-TUNING PROCEDURE

The model was fine-tuned on the Bodo dataset across two progressive stages, running on an NVIDIA T4 GPU via Kaggle.

Dataset split. The dataset was divided into 3,500 training images, 2,826 validation images, and 2,830 test images. Training was carried out on Kaggle using a T4 GPU.

Table 3.1: Fine-Tuning Hyperparameters

Setting	Value
Base checkpoint	microsoft/trocr-base-printed
Bodo tokenizer vocabulary	5,000 tokens (BPE)
Training / Val / Test images	3,500 / 2,826 / 2,830
Optimiser	AdamW
Learning rate	5×10^{-5}
Precision	FP16 mixed
Gradient checkpoint	Enabled
Maximum label sequence length	64 tokens
Stage 1 epochs	5 (warm-up)
Stage 2 epochs	10 (main training)
Beam size	4
no_repeat_ngram_size	3
Hardware	Kaggle T4 GPU

Stage 1 — Warm-up (5 epochs). The first stage comprised 5 warm-up epochs; performance at this stage remained poor, as reflected in Table 3.2, which records the training and validation loss for every epoch.

Table 3.2: Stage 1 (5 Warm-up Epochs)

Epoch	Training Loss	Validation Loss
1	28.555	13.520
2	25.739	12.530
3	23.107	11.519
4	19.489	10.801
5	16.194	10.748

Stage 2 — Main training (10 epochs). After the warm-up stage, training was extended by 10 additional epochs on the same 5,000-sample dataset to further reduce loss and improve recognition accuracy. Table 3.3 records the training and validation loss for every epoch.

Table 3.3: Stage 2 Loss Training History

Epoch	Training Loss	Validation Loss
1	17.280	11.227
2	15.909	10.512
3	12.881	9.891
4	9.707	9.495
5	7.673	9.192
6	5.094	9.114
7	3.851	9.145
8	2.596	9.043
9	2.231	9.092
10	2.104	9.116

Stage 3 — Fine Refinement (Epochs 16–25) During the third stage, the training loss continued to decrease and reached its minimum value of 1.109 at epoch 23. However, the validation loss showed very little improvement, remaining between 9.19 and 9.39, and slightly increasing compared to Stage 2. This difference between training and validation loss indicates *overfitting*, where the model started learning the training data too closely instead of generalising well to unseen Bodo text images. Therefore, the checkpoint obtained in Stage 2 (epoch 13, validation loss = 9.043) was selected for final evaluation and testing because it provided the best overall generalisation performance.

Table 3.4: Stage 3 Loss History (10 Fine Refinement Epochs)

Epoch	Training Loss	Validation Loss
16	2.193	9.187
17	1.679	9.247
18	1.793	9.225
19	1.643	9.257
20	1.475	9.394
21	1.351	9.355
22	1.382	9.316
23	1.109	9.344
24	1.230	9.347
25	1.343	9.358

4. EXPERIMENTAL RESULTS

4.1 EXPERIMENTAL RESULTS OF TESSERACT OCR

Tesseract produced satisfactory results when fine-tuned on the 5,000-sample synthetic dataset. The following figures present input Bodo script images alongside their corresponding Tesseract OCR output.

Input	Output
गिबि खेब हादोर गेजेरारि बड'लेण्ड पेरागलाइडिं सेम्फियनसिफ बादायलायनाया गोजोनै गोजोन जोबलांबाय	गिबि खेब हादोर गेजेरारि बड'लेण्ड पेरागलाइडिं सेम्फियनसिफ बादायलायनाया गोजोनै गोजोन जोबलांबाय

Figure 4.1: Tesseract OCR Result — Sample 1: Short Bodo sentences (printed script).

Input	Output
खनगोरा खोरिया बोराय । सोर्गोयाव दङ दोहोरोमनि मोदाय फाथालाव दङ थुलुसि गोथार बेनिफ्राय सिबिजाथाव रावबो गैथारलिया ।	खनगोरा खोरिया बोराय । सोर्गोयाव दङ दोहोरोमनि मोदाय फाथालाव दङ थुलुसि गोथार बेनिफ्राय सिबिजाथाव रावबो गैथारलिया ।

Figure 4.2: Tesseract OCR Result — Sample 2: Multi-line Bodo paragraph (printed script).

Input

दाउज्ला गिसिरबाय अखा नाइनसै
 हाथाखिं हालाफरा गमरबदंसै;
 सिखिरि सिख्ला सिखांनानै
 दुमदुम-दामदाम माइ सौदंसै।
 ब्लाउ ब्लाउ गजं बनजार धरसेनि,
 साथिबाय जेरावब अलंबारनि
 बिबारबारिआव बाइदि फाखा-सिखा,
 गुफुर गजा गावलांबाय बिबारआ।
 दंफां-लाइफां रुम्बां-रुम्बां
 मस्तित-मस्तात् नुहां-नुवाहां।
 जादंसै हनै अं-अं बेरे मदै,
 बिबार सबदं गदै बिदै।
 दाउस्रि-दाउलाफ्रा मेथाइ खनबाय,
 दंफां-दंफां रंजाबायबाय।
 सरांबाय हनै सरांथारबाय,
 सानजाहा जारौ-रौ सानआ नुजाबबाय।
 सेरेयै थाबाय नों बेसेबा गबाव,
 गदहाब उन्दुनानै खमसि लैथआव।
 जानाय नंलिआ थानाय गद गथौ;
 सिखारद आफा नड बरनि बाथौ।

Output

दाउज्ला गिसिरबाय अखा नाइनसै
 हाथाखिं हालाफरा गमरबदंसै;
 सिखिरि सिख्ला सिखांनानै
 दुमदुम-दामदाम माइ सौदंसै।
 ब्लाउ ब्लाउ गजं बनजार धरसेनि,
 साथिबाय जेरावब अलंबारनि
 बिबारबारिआव बाइदि फाखा-सिखा,
 गुफुर गजा गावलांबाय बिबारआ।
 दंफां-लाइफां रुम्बां-रुम्बां
 मस्तित-मस्तात् नुहां-नुवाहां।
 जादंसै हनै अं-अं बेरे मदै,
 बिबार सबद गदै बिदै।
 दाउस्रि-दाउलाफ्रा मेथाइ खनबाय,
 दंफां-दंफां रंजाबायबाय।
 सरांबाय हनै सरांथारबाय,
 सानजाहा जारौ-रौ सानआ नुजाबबाय।
 सेरेयै थाबाय नों बेसेबा गबाव,
 गदहाब उन्दुनानै खमसि लैथआव।
 जानाय नंलिआ थानाय गद गथौ;
 सिखारद आफा नड बरनि बाथौ।

Figure 4.3: Tesseract OCR Result — Sample 3: Long-form Bodo text passage (printed script).

Although Tesseract produced satisfactory results on the 5,000-sample training set, it worked reliably only on the synthetic printed text (typically passages of 5–10 lines) and was unable to predict raw handwritten samples. This motivated the adoption of the transformer-based TrOCR model, which is presented in the following sections.

4.1.1 Tesseract Failure on Handwritten Input

A key limitation of the fine-tuned Tesseract model is its complete inability to recognise handwritten Bodo script. Figure 4.4 demonstrates this failure: when presented with a handwritten Bodo sample, the Tesseract engine either produces garbled output, random characters, or empty output entirely. This is a direct consequence of training Tesseract exclusively on synthetic printed images — the model was never exposed to the natural variations in stroke width, character slant, and ligature style that are present in handwritten text.

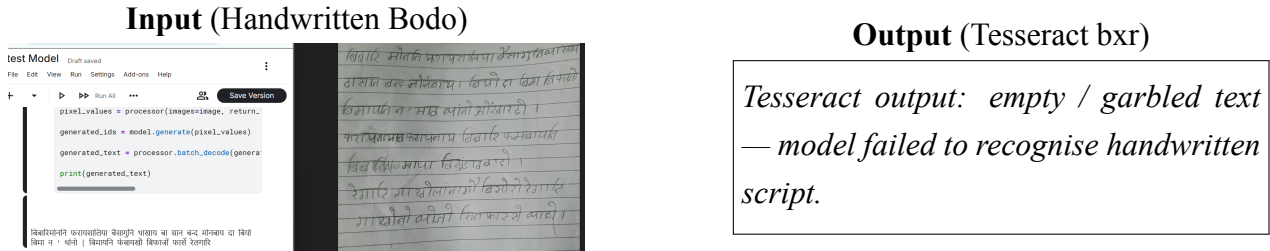


Figure 4.4: Tesseract failure case: the fine-tuned bxr model produces no meaningful output when given a handwritten Bodo script image, highlighting its limitation to printed synthetic text only.

This failure directly motivated the adoption of TrOCR as the primary OCR model for this project, since TrOCR was trained on real handwritten samples and successfully recognised them, as shown in Section 4.5.

4.2 EXPERIMENTAL RESULTS FOR TROCR

The experiments were carried out on a Kaggle notebook with an NVIDIA T4 GPU (16 GB VRAM), 13 GB RAM, and approximately 77 GB of disk storage. The model was trained using Python 3.10, PyTorch 2.0, and the Hugging Face Transformers library.

The dataset was divided into three portions: 3,500 images for training, 2,826 for validation, and 2,830 for testing. All splits contained printed and handwritten Bodo script samples written in Devanagari Unicode.

4.3 EVALUATION METRICS

The two standard OCR evaluation metrics were used for model performance.

Character Error Rate (CER). CER measures the proportion of characters that were incorrectly predicted relative to the total number of characters in the ground truth. It is computed as:

$$CER = \frac{S + D + I}{N} \times 100\% \quad (4.1)$$

where S is the number of character substitutions, D the number of deletions, I the number of insertions, and N the total number of characters in the reference string. A lower CER indicates better performance.

Word Error Rate (WER). WER applies the same edit-distance logic at the word level rather than the character level:

$$WER = \frac{S_w + D_w + I_w}{N_w} \times 100\% \quad (4.2)$$

where the subscript w denotes word-level counts. WER is typically higher than CER and is more sensitive to segmentation errors.

4.4 TRAINING LOSS ANALYSIS

The TrOCR model was fine-tuned in three stages with 25 epochs. Figure 4.5 shows the training and validation loss curves across all three stages.

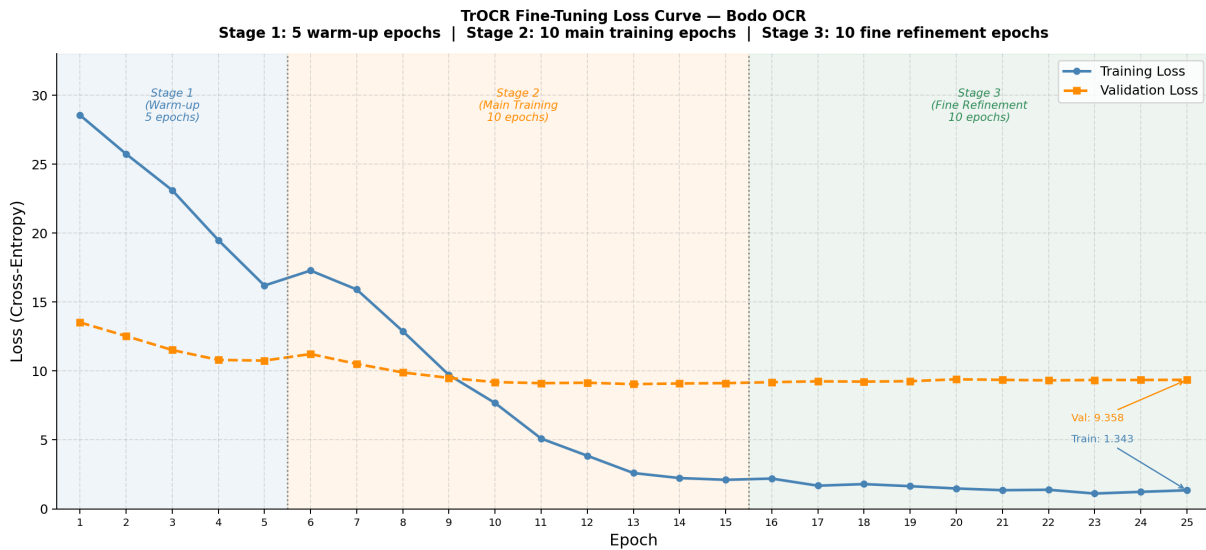


Figure 4.5: TrOCR fine-tuning loss curve for the Bodo OCR model across all three training stages (25 total epochs).

Training: Stage 1 (Epochs 1–5). During the first stage, training was run for 5 epochs as a warm-up. The training loss decreased from 28.56 to 16.19, and the validation loss reduced from 13.52 to 10.75, indicating that the model was beginning to learn and adapt to the Bodo text data. The gap between training and validation loss during this stage was expected, since the model was still in its initial stabilising phase.

Training: Stage 2 (Epochs 6–15). In the second stage, the training loss dropped to 2.10 by epoch 15. The validation loss reached **9.043 at epoch 13** and remained flat thereafter, which was treated as the convergence point. The best model checkpoint was automatically saved at this epoch using the `load_best_model_at_end` flag.

Training: Stage 3 (Epochs 16–25). In the third stage, the training loss continued to decrease and reached its lowest value of 1.109 at epoch 23. However, the validation loss remained almost unchanged, ranging between 9.19 and 9.39. This indicates that the model began overfitting — memorising the training data rather than generalising to unseen Bodo text images. Despite the overfitting, the model was evaluated on the test set and produced satisfactory results.

Table 4.1: Summary of Training Loss Across All Three Stages

Stage	Epochs	Initial Train Loss	Final Train Loss	Best Val Loss	Best Epoch
Stage 1 (Warm-up)	1–5	28.556	16.194	10.748	5
Stage 2 (Main Training)	6–15	17.280	2.104	9.043	13
Stage 3 (Fine Refinement)	16–25	2.193	1.343	9.187	16
Overall	1–25	28.556	1.343	9.043	13

4.5 QUALITATIVE RESULTS OF TROCR

The fine-tuned TrOCR model produced strong qualitative results on the Bodo script test set. It was able to recognise raw handwritten samples, stylised cover-page text on Bodo books, and printed paragraphs ranging from 1 to 8 lines. Representative outputs are shown below.

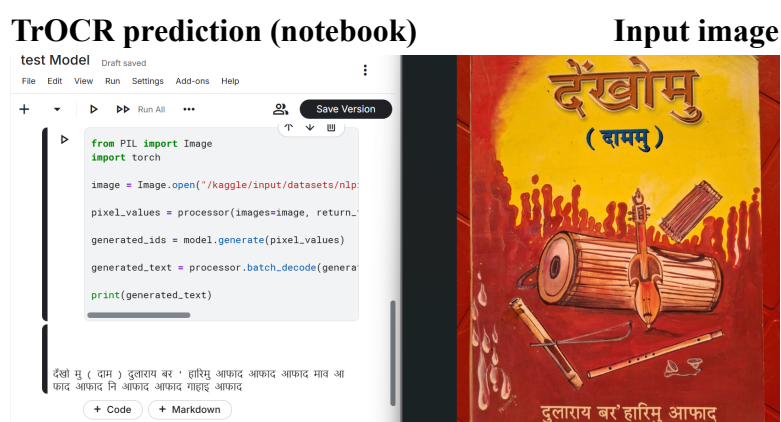


Figure 4.6: TrOCR result on a Bodo book cover. The Kaggle notebook on the left shows the predicted Bodo text; the image on the right is the original input.

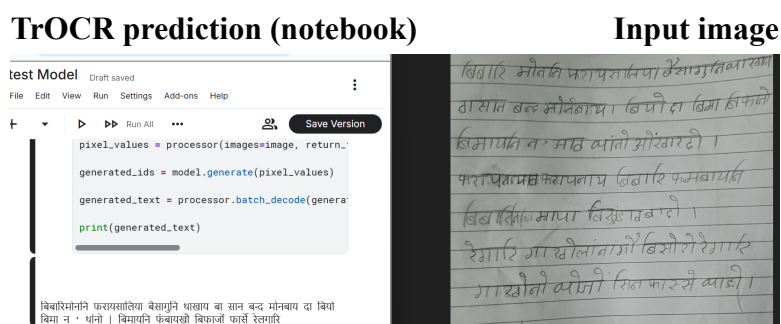


Figure 4.7: TrOCR result on a raw handwritten Bodo sample. The notebook on the left shows the predicted Bodo text; the scan on the right is the original handwritten input.

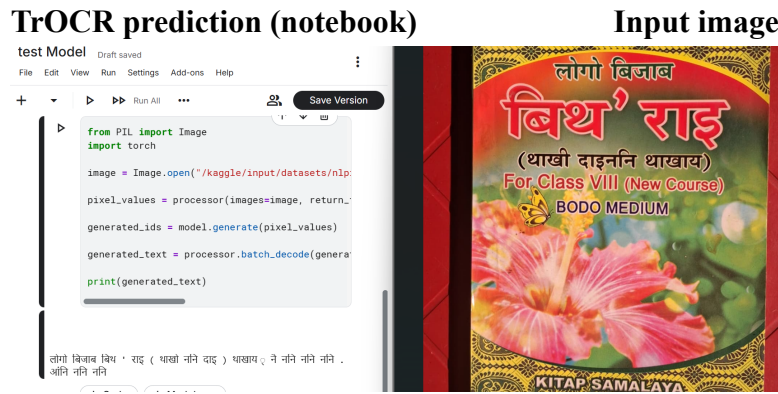


Figure 4.8: TrOCR result on a multi-line Bodo book cover. The notebook on the left shows the predicted Bodo text; the cover on the right is the original input.

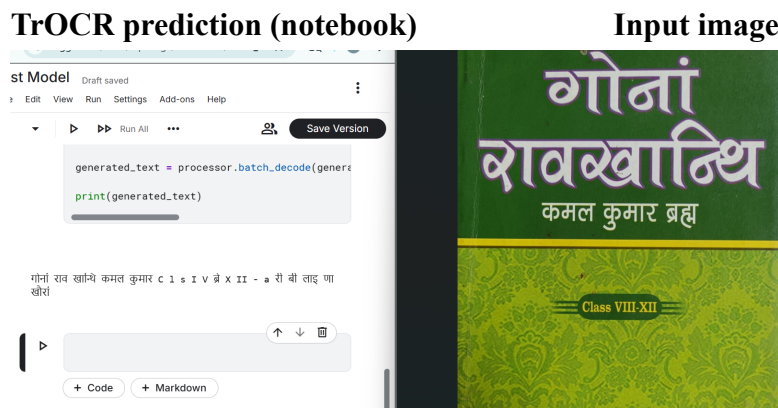


Figure 4.9: TrOCR result on a stylised Bodo book cover. The notebook on the left shows the predicted Bodo text; the cover on the right is the original input.

4.6 DISCUSSION

The results show that the Tesseract brx model can recognise Bodo words on printed input. Page segmentation mode 3, used for full-page segmentation, was effective on the third sample image, and the model handled different font weights and sizes, as illustrated by the second sample. The line-finding stage was also able to segment and recognise text in the presence of blur noise on the input image. Overall, Tesseract OCR is reliable for printed Bodo text and can be fine-tuned for low-resource languages with a custom dataset, even on local CPU or low-end devices.

In contrast, the fine-tuning curves show that the pre-trained microsoft/trocr-base-printed model adapted to the Devanagari-based Bodo character set within just the five warm-up epochs of Stage 1, as evidenced by the steep initial drop in both training and validation loss. Convergence at Stage 2, epoch 13 (validation loss = 9.043) suggests that the model reached its best generalisation capacity given the current training set of 3,500 images.

The overfitting observed in Stage 3 indicates that further improvement would likely require a larger and more diverse training set, stronger regularisation (for example, increased dropout or label smoothing), or data augmentation strategies tailored to Bodo script — such as synthetic skewing, blurring, or noise injection to simulate real-world scanning artefacts.

The custom 5,000-token Bodo BPE tokenizer also proved essential to the model’s success, since it provided proper coverage of the Bodo Unicode character set. During inference, beam search and

repetition control were used to produce cleaner, more accurate output, particularly for longer text lines.

4.7 COMPARATIVE ANALYSIS: TESSERACT VS TROCR

This section presents a structured comparison of the two OCR systems evaluated in this project — the fine-tuned Tesseract LSTM engine and the fine-tuned TrOCR transformer model.

Training Approach

Tesseract was fine-tuned using a synthetic dataset of over 5,000 images generated by a custom Python augmentation pipeline, using the Hindi pre-trained model (`hin.traineddata`) as the base. Training ran for up to 10,000 iterations and halted at iteration 6,100 when the target error rate was reached.

TrOCR was fine-tuned on 5,000 real-world Bodo images selected from a 28,000-sample dataset collected from Bodosia newspaper, Bodo books, and handwritten student samples. Training ran for 25 epochs across three progressive stages on a Kaggle T4 GPU using AdamW with a learning rate of 5×10^{-5} .

Table 4.2: Overall Comparison: Tesseract vs TrOCR for Bodo OCR

Criterion	Tesseract	TrOCR
Architecture	Traditional LSTM pipeline	Encoder-decoder Trans-former
Training data	Synthetic printed images	Real-world printed + hand-written
Training samples	5,000 synthetic	5,000 of 28,000 real images
Printed text	Good	Good to excellent
Handwritten text	Failed completely	Successful
Overfitting	None	Observed in Stage 3
Hardware needed	CPU sufficient	GPU required
Best suited for	Printed documents only	All Bodo script types

4.7.1 Quantitative Comparison: Tesseract vs TrOCR

The quantitative evaluation was conducted on 100 samples from the test set using Character Error Rate (CER) and Word Error Rate (WER). The results are presented in Table 4.3.

Table 4.3: CER and WER on Bodo Test Set (100 samples)

Model	CER (%)	WER (%)
Tesseract (fine-tuned, brx)	4.73	6.50
TrOCR (fine-tuned, Bodo)	287.39	287.39

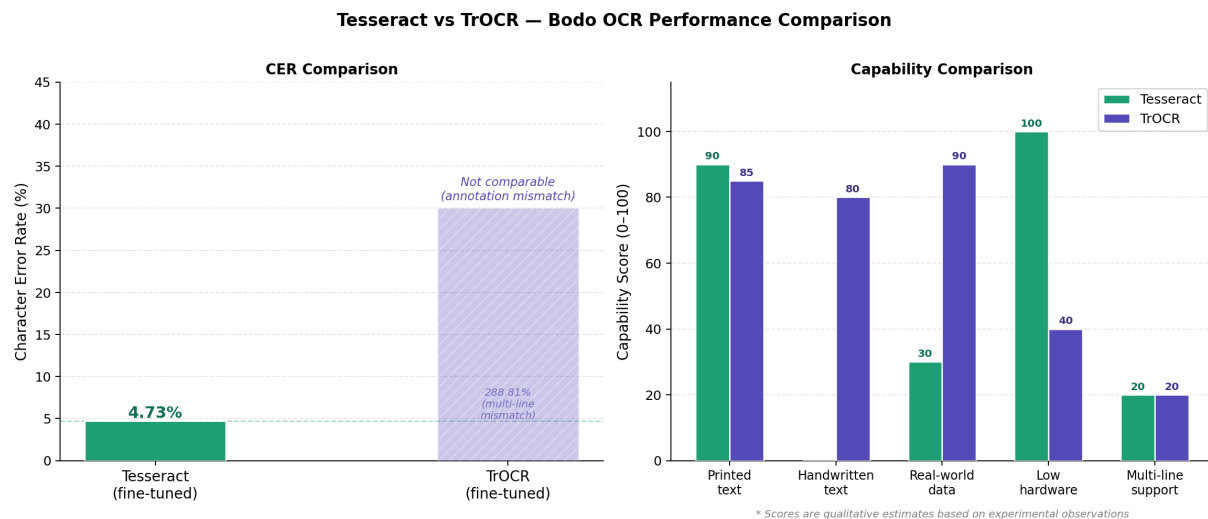


Figure 4.10: Comparative performance of Tesseract and TrOCR for Bodo OCR. Left: CER comparison — Tesseract achieved 4.73% CER on single-line printed text while TrOCR’s CER is not directly comparable due to multi-line annotation mismatch. Right: qualitative capability comparison across five evaluation dimensions based on experimental observations.

The high CER and WER values observed in this evaluation do not directly reflect the model’s recognition capability on individual lines. The test set annotations contain multi-line paragraph-level ground truth text, while the TrOCR model was fine-tuned and optimised for single-line text recognition with a maximum output length of 64 tokens. When a multi-line image is passed to the model, it can only generate a partial prediction covering the first line or portion of the text, while the ground truth spans multiple lines. This structural mismatch between the annotation format and the model’s output capacity causes artificially inflated CER and WER scores.

The qualitative results presented in Section 4.5 demonstrate that the model correctly recognises individual Bodo text lines, including printed newspaper text, book covers, and handwritten samples. A fair quantitative evaluation would require re-annotating the test set at the single-line level, which is recommended as part of future work.

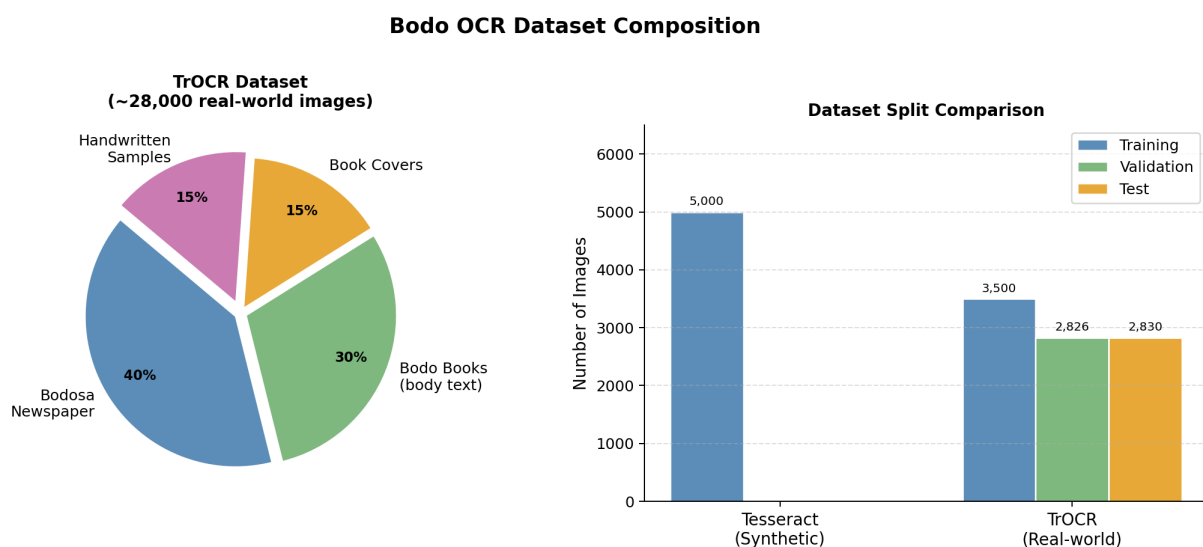


Figure 4.11: Bodo OCR dataset composition. Left: distribution of the four source types in the TrOCR real-world dataset (28,000 images). Right: training, validation, and test split comparison between the Tesseract synthetic pipeline and the TrOCR real-world pipeline.

Key Observations

Both models performed reasonably well on printed Bodo text. However, Tesseract failed entirely on handwritten samples because it was trained only on synthetic printed images and was never exposed to handwriting during fine-tuning. TrOCR successfully recognised both printed and handwritten Bodo text, including stylised book cover text and natural student handwriting, owing to its real-world training data and the global attention mechanism of its ViT encoder.

Tesseract showed stable training convergence with a final CER of approximately 1.880% at iteration 6,100. TrOCR reached its best generalisation at Stage 2 epoch 13 (validation loss = 9.043) and showed signs of overfitting in Stage 3, where training loss continued to fall while validation loss plateaued. Despite this, qualitative results on the test set remained satisfactory.

Overall, TrOCR is the superior approach for Bodo OCR. Even when trained on only 5,000 samples, it outperformed Tesseract on printed text and decisively surpassed it on handwritten input. Training TrOCR on the full 28,000-sample dataset is expected to further reduce overfitting and improve recognition accuracy.

5. CONCLUSION

In this project, we conducted an experimental comparative study of two OCR methods optimised for Bodo script recognition: the LSTM-based Tesseract OCR engine and the transformer-based TrOCR model developed by Microsoft Research [1]. To the best of our knowledge, this is one of the first comprehensive investigations of OCR for the Bodo language.

For Tesseract OCR, the model was fine-tuned from the Hindi base model under the ISO 639-3 code `bxr`, using more than 5,000 synthetic training images produced by our custom Python script. For TrOCR, a set of 28,000 real-world images with text annotations was assembled from the Bodosa newspaper, Bodo books and covers, and handwritten student samples. Within the available compute budget, 5,000 of these samples were used to fine-tune TrOCR across three progressive stages totalling 25 epochs.

According to the experimental results, Tesseract worked adequately on printed Bodo text but failed to recognise handwritten samples. TrOCR outperformed Tesseract on printed text, and decisively so on handwritten input, even when only 5,000 of the 28,000 available images were used for fine-tuning. On this evidence, TrOCR is the better-suited method for Bodo OCR.

5.1 DRAWBACKS

- **Limited training samples.** Due to memory and GPU constraints on the Kaggle T4 environment, only 5,000 of the available 28,000 images were used for fine-tuning TrOCR. This directly contributed to the overfitting observed in Stage 3 and limited the model's ability to generalise to the full diversity of Bodo script.
- **Hardware limitation.** The Kaggle free-tier T4 GPU restricted the batch size, training duration, and total number of samples that could be processed. A larger GPU environment would allow training on the complete dataset and experimentation with larger model variants such as `trocr-large`.
- **Tesseract cannot handle handwriting.** All samples used to fine-tune Tesseract were synthetic and printed. As a result, the model is essentially unable to recognise handwritten Bodo text, which significantly limits its practical usefulness for real-world digitisation tasks.
- **Multi-line Limitation.** The fine-tuned TrOCR model demonstrated strong performance on single-line Bodo text but struggled with multi-line images. This is a known architectural limitation of the base TrOCR model, which is designed for single text-line recognition. Multi-line document OCR would require a separate page segmentation module to first split the document into individual text lines before passing them to the TrOCR model. This is identified as an important direction for future work.

6. FUTURE WORK

The most immediate improvement would be to fine-tune TrOCR on the full 28,000-sample dataset; this is expected to significantly reduce the overfitting observed in Stage 3 and improve the model’s ability to generalise to unseen Bodo script images. Beyond this, several directions are worth exploring:

- **Line segmentation for multi-line document OCR.** The current TrOCR model is designed for single text-line recognition and cannot process full-page multi-line documents directly. Quantitative evaluation on our test set revealed that the high CER and WER values were primarily caused by a mismatch between the model’s single-line output capacity and the multi-line ground truth annotations. A practical solution would be to add a line segmentation preprocessing step using OpenCV horizontal projection profiling, which splits a full document page into individual line crops before passing each line to TrOCR. This would also allow the full 28,000-image dataset to be converted into over 100,000 single-line training samples, significantly reducing overfitting and improving generalisation. Building this complete document-level pipeline is identified as the most impactful direction for future work.
- **Stronger regularisation and data augmentation.** Applying dropout scheduling and label smoothing during fine-tuning, combined with Bodo-specific data augmentation strategies such as synthetic skewing, Gaussian blurring, and noise injection, would expose the model to a wider variety of image conditions and reduce its tendency to memorise the training set.
- **Post-OCR correction module.** Integrating a dedicated post-processing correction module — for example, a sequence-to-sequence spell-checker trained on Bodo text — would reduce residual character errors that persist even in well-trained models, particularly for rare or out-of-vocabulary Bodo words.
- **Larger model variants.** Experimenting with `trocr-large` instead of `trocr-base` would provide a higher-capacity encoder and decoder, which is expected to improve recognition accuracy for complex Devanagari conjunct consonants and vowel matras that the base model occasionally misclassifies.
- **Document-level OCR pipeline.** Building a complete document-level pipeline with automatic layout analysis and text-line segmentation would enable the digitisation of full Bodo books, newspapers, and archival manuscripts — moving beyond single-line recognition toward a practical, deployable system for Bodo digital preservation.
- **Public dataset release.** Publicly releasing the curated 28,000-sample Bodo script dataset would provide the research community with a valuable resource for future work in Bodo language NLP, low-resource OCR, and Devanagari script processing, helping to accelerate progress in the digitisation and computational study of the Bodo language.

BIBLIOGRAPHY

- [1] M. Li, R. Lyu, L. Cui, S. Huang, Y. Lü, and F. Wei, “TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models,” *arXiv preprint arXiv:2109.10282*, 2021.
- [2] R. L. Zhang, “A Comprehensive Evaluation of TrOCR with Varying Image Effects,” *arXiv preprint*, 2022.
- [3] A. Dosovitskiy *et al.*, “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.
- [4] Hugging Face, “TrOCR Model Documentation,” https://huggingface.co/docs/transformers/en/model_doc/trocr, accessed May 2026.
- [5] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] U. Pal and B. B. Chaudhuri, “A survey on optical character recognition for Bangla and Devanagari scripts,” *Sadhana*, vol. 38, no. 1, pp. 133–168, 2013.
- [7] R. S. Publication, “Optical Character Recognition for Devanagari-based Indian Languages,” *International Journal of Research in Engineering & Science*, 2025.
- [8] H. Kashid and P. Bhattacharyya, “RoundTripOCR: A Data Generation Technique for Enhancing Post-OCR Error Correction in Low-Resource Devanagari Languages,” *arXiv preprint arXiv:2412.15248*, Dec. 2024.
- [9] A. Zohar, S. Tsarfaty, and R. Tsarfaty, “Exploiting Script Similarities to Compensate for the Large Amount of Data in Training Tesseract LSTM: Towards Kurdish OCR,” *Applied Sciences*, vol. 11, no. 20, p. 9752, Oct. 2021.
- [10] D. Frimane, A. Rachid, and M. Fakir, “Advancing Arabic Text Recognition: Fine-tuning of the LSTM Model in Tesseract OCR,” *ResearchGate*, Jul. 2023.
- [11] I. Anuradha, “Adapting the Tesseract Open-Source OCR Engine for Tamil and Sinhala Legacy Fonts and Creating a Parallel Corpus for Tamil-Sinhala-English,” *arXiv preprint arXiv:2109.05952*, Sep. 2021.
- [12] N. Saini *et al.*, “OCR Synthetic Benchmark Dataset for Indic Languages,” *arXiv preprint arXiv:2205.02543*, 2022.
- [13] A. Sarkar *et al.*, “Printed OCR for Extremely Low-Resource Indic Languages,” in *Computer Vision and Image Processing, CVIP 2024*, Communications in Computer and Information Science, vol. 2474, Springer, Cham, 2026.
- [14] SynthOCR-Gen Authors, “SynthOCR-Gen: A Synthetic OCR Dataset Generator for Low-Resource Languages,” *arXiv preprint arXiv:2601.16113*, Jan. 2026.
- [15] L. Ramamoorthy, N. Choudhary, B. Basumatary, and F. Daimary, “A Gold Standard Bodo Raw Text Corpus,” Linguistic Data Consortium for Indian Languages (LDC-IL), Central Institute of Indian Languages, Mysore, 2019. <https://data.ldcil.org/a-gold-standard-bodo-raw-text-corpus>
- [16] “OCR for Bodo Script,” *Journal of Information Systems Engineering and Management (JISEM)*, 2025. <https://jisem-journal.com/index.php/journal/article/view/3391/1461>
- [17] Office of the Registrar General & Census Commissioner, India, “Census of India 2011: Language Data,” Government of India, New Delhi, 2011.
- [18] Government of India, “The Constitution of India: Eighth Schedule — Languages,” Ministry of Law and Justice, New Delhi. <https://legislative.dept.gov.in>